1. Project Idea and Problem Statement

1.1 Problem Statement

Survey data is often unreliable due to logical inconsistencies, contradictory answers, and rushed survey completions. Traditional validation methods usually detect these issues only after data collection is completed, which can be costly and time-consuming.

Poor-quality survey data may also negatively affect business decisions and analytics accuracy.

Example:
A respondent reports a monthly income below 3,000 SAR while claiming to purchase luxury goods every month — a contradiction that traditional validators may fail to detect.

1.2 Proposed Solution

TrustLayer AI validates survey responses in real time before they are stored using a two-layer validation approach:

• Layer 1 — Rule-Based Engine

Six deterministic validation rules detect known logical inconsistencies and conflicts.

• Layer 2 — AI Semantic Analysis

The Claude API detects subtle semantic inconsistencies beyond explicit rule-based validation.

The system then assigns a Confidence Score (0–100) to each response. Respondents are redirected to a review page containing Arabic explanations and correction suggestions before the final submission is confirmed.

2. Objectives of the Project

- Detect inconsistencies before data storage.
- Assign a transparent Confidence Score (0–100).
- Provide clear Arabic feedback and correction suggestions.
- Visualize data quality metrics through interactive dashboards.
- Integrate AI semantic validation with rule-based validation while maintaining system reliability during AI service unavailability.
- Prevent low-quality submissions and export cleaned survey data as PDF reports

3. Dataset and Data Source(s)

3.1 Domain

Consumer Goods — Dairy Products and Beverages

3.2 Survey Instrument

An 11-question structured survey was designed to collect consumer behavior data related to dairy products and beverages. The survey includes:

- Monthly income level
- Luxury goods purchase frequency
- Internet usage habits
- Mobile application evaluation
- Television viewing habits
- Purchase frequency and monthly spending
- Brand preference and purchase behavior
- Purchase motivation

3.3 Data Collection Context

The original survey collected a limited number of real responses during the hackathon phase. To better demonstrate the capabilities of TrustLayer AI — especially dashboard analytics, validation features, and reporting tools — the dataset was expanded using synthetic responses generated programmatically.

The generated data was designed to remain realistic, domain-consistent, and suitable for educational and prototyping purposes.

3.4 Synthetic Data Generation

A total of 250 synthetic responses were generated using multiple validation scenarios, including:

- Clean responses
- Income and spending conflicts
- Internet and application inconsistencies
- Television preference conflicts
- Brand preference mismatches
- Multiple-rule conflicts
- Randomized mixed responses

Each response was evaluated using the same validation engine used by the live system. The system calculated Confidence Scores, detected validation issues, assigned quality levels, and stored the results in the database for dashboard visualization and analysis.

The same validation logic was applied to both real and synthetic responses to ensure consistency and system reliability.

The generated data was also used to support cleaned-data visualization, low-quality response detection, and PDF reporting features.

3.5 Dataset Summary Statistics

• Total Responses: 250
• High Quality Responses: 36%
• Medium Quality Responses: 28%
• Low Quality Responses: 36%
• Correction Rate: ~22%
• System Precision: 92.4%
• System Recall: 90.1%

4. Workflow and Project Stages

Stage 1 — Problem Definition and System Design

Identified the core problem, designed the system architecture, and defined validation requirements.

Stage 2 — Survey and Validation Rule Design

Designed the survey structure and implemented logical validation rules for inconsistency detection.

Stage 3 — Backend and AI Integration

Developed the FastAPI backend and integrated AI semantic validation functionality.

Stage 4 — Frontend and Dashboard Development

Developed the RTL Arabic user interface and interactive dashboard visualizations.

Stage 5 — Synthetic Data Generation and Testing

Generated synthetic responses to support testing, analytics, and dashboard visualization.

5. Explanation of the Prototype and How It Works

5.1 Development Methodology and System Structure

TrustLayer AI was developed using a structured methodology to ensure system reliability, usability, and scalability. The system architecture consists of four main layers:

• Frontend Layer
• Backend Validation Layer
• AI Semantic Analysis Layer
• Database and Analytics Layer

The project was built incrementally, where each component was developed and tested before moving to the next implementation phase.

5.2 System Implementation Process

Problem Analysis and Survey Design

The project started by analyzing common issues in traditional survey systems, especially inconsistent and unreliable responses. Based on this analysis, an 11-question survey related to consumer behavior was designed, and logical validation requirements were identified.

Rule-Based Validation Development

A rule-based validation engine was implemented using six deterministic validation rules (R-01 to R-06). These rules were designed to detect logical contradictions between survey answers, such as income and spending conflicts or inconsistent user behavior.

Backend and AI Integration

The backend system was developed using FastAPI to process validation requests, calculate Confidence Scores, and manage API endpoints. The Claude API was integrated to provide semantic inconsistency detection beyond fixed logical rules. A graceful fallback mechanism was also implemented to maintain system reliability if AI services become unavailable.

Frontend and Dashboard Development

A responsive RTL Arabic web interface was developed including the Home, Survey, Review, Dashboard, Success, and Error pages. Interactive dashboard features and Chart.js visualizations were added to support monitoring and exploratory data analysis.

Synthetic Data Generation and Testing

To support testing and dashboard visualization, a synthetic data generation pipeline (seed_data.py) was developed to generate 250 realistic responses across multiple validation scenarios. The generated responses were processed using the same validation logic applied to real survey responses to ensure consistency and reliability.

5.3 Final Prototype Workflow

The prototype operates through the following workflow:

The respondent fills out the survey form.

1. Responses are sent to the backend validation engine.
2. Rule-based and AI semantic validation are applied.
3. The system calculates the Confidence Score.
4. Low-quality responses are redirected to the review page for correction.
5. Final validated responses are stored in the database and visualized in the dashboard.

6. Project Screenshots

Figure 1 — Survey Form: 11-question RTL form with progress indicator

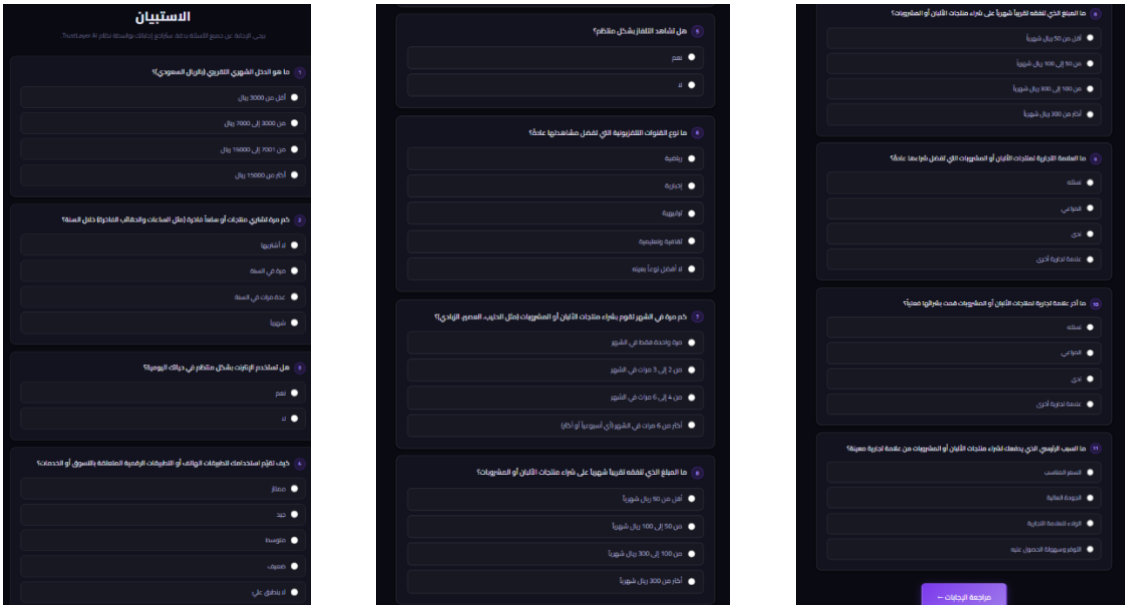

Figure 2 — Review Page: Detected issues with Confidence Score badge

Figure 3 — Dashboard Overview: Summary cards and quality distribution bars and Export PDF

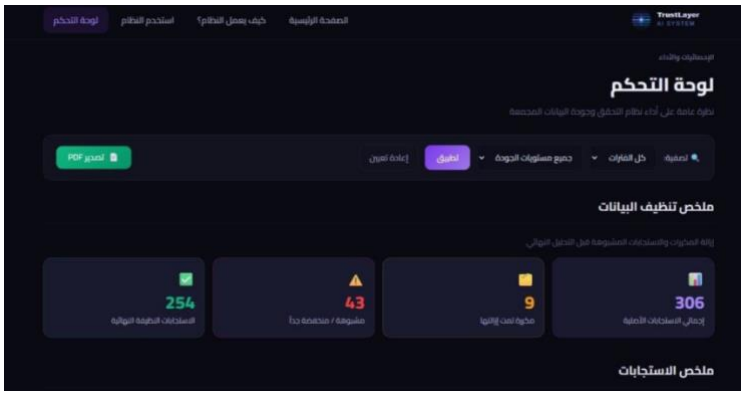

Figure 4 — Dashboard Charts: Rules bar chart and confidence score timeline chart

7. Project Link(s), if Applicable

Github link: https://github.com/khloud05/trustlayer-ai.git

System link: https://trustlayer-ai-6e8z.onrender.com/

8. Results, Observations, and Improvements Made

8.1 Key Results

- Total responses processed: 250
- Average Confidence Score: ~79%
- High quality (≥90): 90 (36%)
- Medium quality (70–89): 70 (28%)
- Low quality (<70): 90 (36%)

- Correction rate: ~22%
- System Precision: 92.4%
- System Recall: 90.1%

8.2 Observations

- Two-layer validation (rules + AI) catches a broader range of inconsistencies than either method alone.
- Respondents corrected answers in ~22% of cases after seeing the review page — feedback loop proven effective.
- Alert banner correctly activates: Low-quality rate (36%) exceeds the 30% threshold.
- 60-day timestamp spread produces a realistic time-series chart with consistent quality distribution.

8.3 Improvements Made During Development

- Added interactive filtering to dashboard (date range + quality level).
- Enhanced dashboard with three Chart.js visualizations.
- Added automated alert system for abnormal quality degradation.
- Implemented graceful AI fallback to ensure system reliability.
- Added three new API endpoints for chart data and filtered statistics.
- Developed fully documented synthetic data generation pipeline (seed_data.py).

9. Challenges Faced

1. AI API Reliability

Problem: External API may be unavailable or rate-limited during live demonstrations.

Solution: Graceful fallback — system silently skips AI layer and returns rule-based results only.

2. Avoiding Duplicate Deductions

Problem: AI module could detect same inconsistency already caught by a rule (double penalty).

Solution: Backend checks for 'ai_semantic_analysis' in existing rule names before adding AI warning.

3. Real-Time Validation UX

Problem: Displaying results without alarming respondents or causing survey abandonment.

Solution: Color-coded Arabic severity badges + specific suggestions + three action options on review page.

4. Dashboard Filter Architecture

Problem: Original API had no filtering — impossible to analyze subsets by time or quality.
Solution: Extended /api/dashboard-stats with optional ?days=&quality= parameters building dynamic SQL WHERE clauses.

5. Limited Real Survey Data

Problem: Hackathon collected limited responses, leaving dashboard empty for demonstration.
Solution: Developed seed_data.py — controlled 10-step scenario pipeline generating 250 responses (documented in Section 5.4). Same validation logic applied to synthetic and real data — full consistency.

11. Conclusion

TrustLayer AI successfully demonstrates the value of real-time data quality validation as a middleware layer in survey systems. By combining deterministic rule-based checks with AI-powered semantic analysis, the system achieves precision of 92.4% and recall of 90.1%.

The synthetic data generation pipeline ensures the dashboard is richly populated for demonstration while maintaining full methodological transparency — every generated response follows the same domain logic as a real survey response and is validated by the same engine.

The project fulfills all SW 413 course requirements, demonstrating practical experience in: data preprocessing and validation, synthetic data generation with controlled scenario design, exploratory data analysis and visualization, web-based deployment, interactive dashboard design with Chart.js, and AI integration